

Corte 1

Repaso y Python

Nathaly Sánchez G.

Universidad Militar Nueva Granada
nubia.sanchez@unimilitar.edu.co

Ingeniería Mecatrónica
18 de febrero de 2026

Contenido I

1 Lenguajes de Programación

- lista de lenguajes

- Python

- control de versiones

2 Fundamentos de programación

- Algoritmos

- Elementos de programación

- Ejercicios

3 Programación Orientada a Objetos (POO)

- Características

- Estructura general de la POO

- Ejercicio

4 Constructor en Python

- Encapsulación

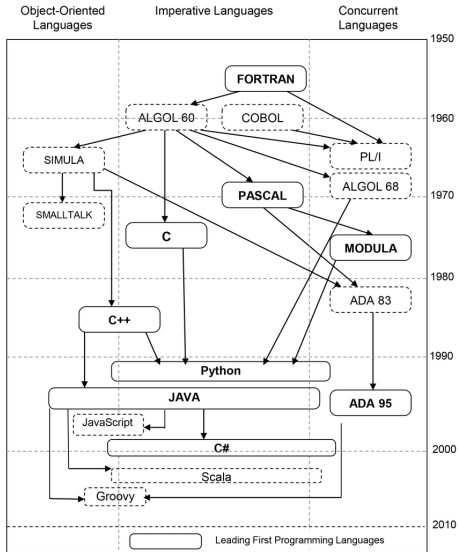
5 Algoritmos de Búsqueda y ordenamiento

- Algoritmos de búsqueda

Contenido II

Algoritmos de ordenamiento

Lenguajes de programación



Software desarrollado con diferentes lenguajes de programación

- 1 C
- 2 C++
- 3 Java
- 4 Python

1



2



3

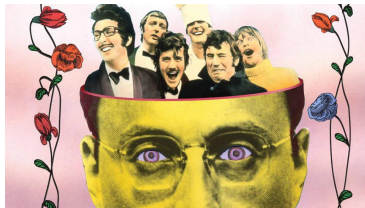


4



Generalidades

PYTHON es un lenguaje de programación creado por Guido van Rossum a principios de los 90 e inspirado en el grupo de cómicos ingleses "Monty Python".



Se trata de un lenguaje interpretado o de script, con tipado dinámico, fuertemente tipado, multiplataforma y orientado a objetos

Su sintaxis es simple, clara y sencilla.

- Se cuenta con gran variedad de librerías disponibles
- Su sintaxis es sencilla y cercana al lenguaje natural.
- No es muy recomendado para la programación de bajo nivel o para aplicaciones en las que el rendimiento sea crítico.

Existen varias implementaciones de Python: CPython, Jython, IronPython, PyPy.

- CPython es la más utilizada, la más rápida y la más madura. Está instalada en la mayor parte de las distribuciones Linux y en las últimas versiones de macOS. En este caso, tanto el intérprete como los módulos están escritos en C.
- Jython es la implementación en Java de Python.
- IronPython es su contrapartida en C# (.NET).
- PyPy es una implementación de Python escrita en Python.

La descarga se puede realizar en: python.org

Control de versiones de Código

Existen sistemas de versionamiento de código que permiten hacer un “tracking” de cada una de las actualizaciones o nuevos registros que se realicen en el código.

El versionamiento permite generar el código de manera distribuida y le permite a equipos de desarrollo trabajar de manera simultánea en desarrollo de grandes sistemas de información.



Algoritmos

- “Conjunto ordenado y finito de operaciones que permite hallar la solución de un problema”. (RAE)
- Secuencia de pasos que resuelven un problema.
- Descripción de un procedimiento paso a paso, el cual da solución a un problema específico.

```

Algoritmo Área.Rectángulo2
  Escribir 'CÁLCULO DEL ÁREA DE UN RECTÁNGULO'
  Repetir
    Escribir 'Teclea la base(en centímetros):'
    Leer base
  Hasta Que base>0
  Escribir 'Teclea la altura(en centímetros):'
  Leer altura
  área <- base*altura
  Escribir 'Área = ',área,' centímetros cuadrados.'
FinAlgoritmo

```

Figura: Pseudocódigo

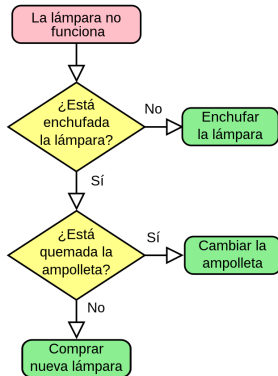


Figura: D. flujo

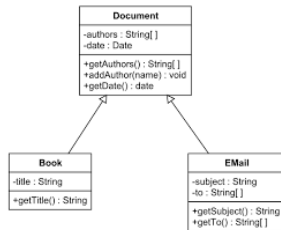


Figura: D. UML

Elementos de programación

Datos

Primitivos

Variables miembro

Referencia

Variables locales

Operadores

Operadores Matemáticos

Operadores Lógicos

Conectores Lógicos

Funciones

Operaciones propias que dan solución a un problema específico

Sentencias de control

Sentencias de selección

Sentencias Repetitivas

Ejercicio 1

Write a program that asks the user for a positive nonzero integer value. The program should use a loop to get the sum of all the integers from 1 up to the number entered. For example, if the user enters 50, the loop will find the sum of 1, 2, 3, 4, . . . , 50.

Ejercicio 2

Write an `if-else` statement that assigns 0 to `x` when `y` is equal to 10. Otherwise it should assign 1 to `x`.

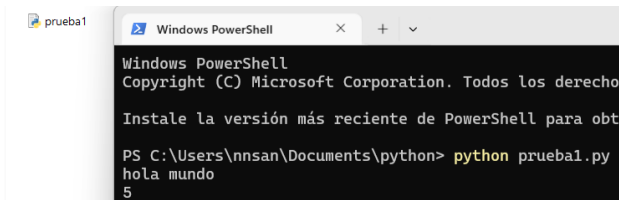
Python

Existen dos formas de ejecutar código Python:

Podemos escribir líneas de código en el intérprete y obtener una respuesta del intérprete para cada línea (sesión interactiva)

```
C:\Users\nnsan>python
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> print("Hola mundo")
Hola mundo
>>> x=2
>>> y=3
>>> z=x+y
>>> z
5
```

Podemos escribir el código de un programa en un archivo de texto y ejecutarlo.

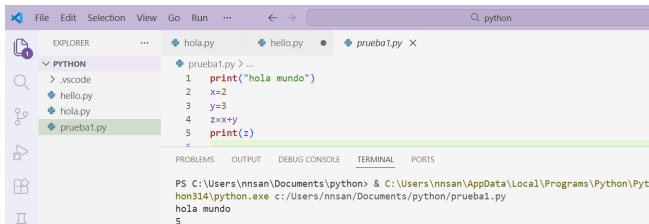
A screenshot of a Windows PowerShell window. The title bar shows 'prueba1' and 'Windows PowerShell'. The window contains the following text: 'Windows PowerShell', 'Copyright (C) Microsoft Corporation. Todos los derechos reservados.', 'Instale la versión más reciente de PowerShell para obtener las mejores características.', and a command prompt 'PS C:\Users\nnsan\Documents\python> python prueba1.py' followed by the output 'hola mundo' and '5'.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener las mejores características.

PS C:\Users\nnsan\Documents\python> python prueba1.py
hola mundo
5
```

Visual studio code



Comentario de una sola línea

```
# Este es un comentario de una sola línea  
x = 10  # Comentario al final de la línea
```

Comentario de múltiples líneas

```
"""
```

```
Este es un comentario  
de múltiples líneas.
```

```
"""
```


Ejercicios

Escriba un programa en Python que solicite al usuario 4 números enteros para calcular su suma y evalúe los siguientes rangos en el resultado:

- Está en el rango de **0 a 5**.
- Está en el rango de **6 a 10**.
- Es **mayor que 10**.

Ejercicios

```
ejercicio.py > ...
1 suma = 0
2
3 for i in range(4):
4     numero = int(input("Ingrese un número entero: "))
5     suma += numero
6
7 print("La suma es:", suma)
8
9 if 0 <= suma <= 5:
10     print("La suma está entre 0 y 5")
11 elif 6 <= suma <= 10:
12     print("La suma está entre 6 y 10")
13 else:
14     print("La suma es mayor que 10")
15
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
Ingrese un número entero: 1
Ingrese un número entero: 2
Ingrese un número entero: 3
Ingrese un número entero: 4
La suma es: 10
La suma está entre 6 y 10
```

Ejercicios

Desarrolle un programa en Python que solicite al usuario su categoría en la caja de compensación (A, B o C) para determinar el costo de la actividad recreativa:

- Categoría A: 3 dólares.
- Categoría B: 6 dólares.
- Categoría C: 10 dólares.

Use un ciclo para permitir que el programa se ejecute de forma repetitiva. Finalice el programa cuando el usuario lo indique.

Ejercicios

```
ejercicio.py > ...
1  while True:
2      categoria = input(
3          "Ingrese su categoría (A, B, C) o X para salir: "
4      )
5
6      match categoria:
7          case "A":
8              print("Categoría A")
9              print("Costo de la actividad: 3 dólares")
10         case "B":
11             print("Categoría B")
12             print("Costo de la actividad: 6 dólares")
13         case "C":
14             print("Categoría C")
15             print("Costo de la actividad: 10 dólares")
16         case "X":
17             print("Programa finalizado.")
18             break
19         case _:
20             print("Categoría no válida.")
21
22     print()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

14\python.exe c:/Users/nnsan/Documents/python/ejercicio.py
Categoría no válida.

Ingrese su categoría (A, B, C) o X para salir: A
Categoría A
Costo de la actividad: 3 dólares

Funciones

Se define con la palabra reservada **def**.

```
def saludar():  
    print("Hola mundo")  
saludar()
```

```
def acelerar(self):  
    self.velocidadActual *= 2
```

```
imprimir.py > ...  
1  
2 def imprimir(texto, veces):  
3     print(texto * veces)  
4  
5 imprimir("hola", 2)
```

PROBLEMS OUTPUT DEBUG CONSOLE TER

```
PS C:\Users\nnsan\Documents\python>  
imir.py  
holahola
```

Funciones

- Funciones con parámetros:

```
def sumar(a, b):  
    resultado = a + b  
    print(resultado)
```

```
sumar(5, 3)
```

- Funciones con retorno:

```
def multiplicar(a, b):  
    return a * b
```

```
res = multiplicar(4, 2)  
print(res)
```

Programación orientada a objetos

Características

- “Todo es un objeto”
- Un programa es un cúmulo de objetos que se dicen entre sí lo que tienen que hacer mediante el envío de señales
- Todo objeto es de algún tipo (cada objeto es un elemento de una clase)

Estructura general de la POO

Clase

Generalización de los objetos/ Categorías de los objetos

Objeto

Representan cosas físicas o abstractas que tienen un estado y un comportamiento.

Atributo

Valores o características de los objetos

Permiten definir el estado del objeto u otras cualidades

Método

Acciones que puede realizar un objeto

Mensaje

Formas de interacción o comunicación entre objetos

Clases en Python

Una clase es un molde para crear objetos.

```
class Estudiante:
    def __init__(self, nombre, edad):
        self.nombre = nombre
        self.edad = edad

    def saludar(self):
        print("Hola, soy", self.nombre)
```

- `__init__` es el constructor.
- `self` representa al objeto.

Objetos en Python

Un objeto es una instancia de una clase.

```
est1 = Estudiante("Ana", 20)  
est2 = Estudiante("Luis", 22)
```

```
est1.saludar()  
est2.saludar()
```

```
print(est1.edad)
```

Cada objeto tiene sus propios datos y comportamientos.

Ejercicio

Implementa la clase `Bicicleta`, que tiene tres atributos, `velocidadActual`, `platoActual` y `piñonActual`, de tipo entero y cuatro métodos `acelerar()`, `frenar()`, `cambiarPlato(int plato)`, y `cambiarPiñon(int piñon)`, donde el primero dobla la velocidad actual, el segundo reduce a la mitad la velocidad actual, y el tercero y cuarto ajustan el plato y el piñón actual respectivamente según los parámetros recibidos. La clase debe tener además un constructor que inicialice todos los atributos.

- Crea dos objetos de la clase bicicleta: `miBicicleta` y `tuBicicleta`

Ejercicios

```
Bici.py > ...
1  class Bicicleta:
2      def __init__(self, velocidadActual, platoActual, pinonActual):
3          self.velocidadActual = velocidadActual
4          self.platoActual = platoActual
5          self.pinonActual = pinonActual
6
7      def acelerar(self):
8          self.velocidadActual *= 2
9
10     def frenar(self):
11         self.velocidadActual //= 2
12
13     def cambiarPlato(self, plato):
14         self.platoActual = plato
15
16     def cambiarPinon(self, pinon):
17         self.pinonActual = pinon
18
19
20     # Creación de objetos
21     miBicicleta = Bicicleta(10, 2, 5)
22     tuBicicleta = Bicicleta(8, 1, 4)
```

Ejercicio1

```
+static void main(String[] args)
```

Bicicleta

```
-int velocidadActual
```

```
+int platoActual
```

```
+int pinonActual
```

```
◆ +Bicicleta(int velocidadActual, int platoActual, int pinonActual)
```

```
● +void acelerar()
```

```
● +void frenar()
```

```
● +void cambiarPlato(int platoActual)
```

```
● +void cambiarPinon(int pinonActual)
```

Particularidades del constructor en Python

- El constructor en Python se llama `__init__`
- Se ejecuta automáticamente al crear un objeto
- No tiene valor de retorno
- Siempre recibe como primer parámetro a `self`
- Sirve para inicializar los atributos del objeto
- Puede recibir parámetros
- Puede usar valores por defecto
- No existe sobrecarga de constructores como en Java o C++
- Es público por defecto (Python no usa modificadores de acceso)

Ejemplo de constructor en Python

```
def __init__(self, velocidadActual, platoActual, pinonActual)
    self.velocidadActual = velocidadActual
    self.platoActual = platoActual
    self.pinonActual = pinonActual
```

- `__init__` se ejecuta al crear el objeto
- `self` representa al objeto creado
- Los atributos se inicializan dentro del constructor

Encapsulación

La encapsulación se refiere a impedir el acceso a determinados métodos y atributos de los objetos estableciendo así qué puede utilizarse desde fuera de la clase.

En Python no existen los modificadores de acceso, y lo que se suele hacer es que el acceso a una variable o función viene determinado por su nombre:

si el nombre comienza con dos guiones bajos (y no termina también con dos guiones bajos) se trata de una variable o función privada, en caso contrario es pública. Los métodos cuyo nombre comienza y termina con dos guiones bajos son métodos especiales que Python llama automáticamente bajo ciertas circunstancias.

```
Ejemplo.py > ...  
1  class Ejemplo:  
2      def publico(self):  
3          print ("Publico")  
4      def __privado(self):  
5          print ("Privado")  
6  ej = Ejemplo()  
7  ej.publico()  
8  ej.__privado()
```


Sentencias de control de flujo

- Condicional if:

```
edad = 18
if edad >= 18:
    print("Es mayor de edad")
```

- condicional if-else:

```
nota = 3.2
if nota >= 3:
    print("Aprobado")
else:
    print("Reprobado")
```

- Condicional if - elif - else:

```
temperatura = 25
if temperatura < 10:
    print("Frio")
elif temperatura < 25:
    print("Templado")
```

Sentencias de control de flujo

- ciclo for:

```
for i in range(5):  
    print(" Iteracion :", i)
```

- Ciclo for en Python: rango y pasos

```
for i in range(5, 11):  
    print(i)
```

- for que avanza de dos en dos:

```
for i in range(0, 11, 2):  
    print(i)
```

Sentencias de control de flujo

- Ciclo while:

```
contador = 0
```

```
while contador < 5:  
    print(contador)  
    contador += 1
```

Recursividad

Una función es recursiva cuando se llama a sí misma.

Se usa para:

- Problemas repetitivos
- Estructuras jerárquicas

Cadenas de texto (String) en Python

Ejemplo:

```
texto = "Hola mundo"  
print(texto)
```

Acceso y longitud de caracteres:

```
print(texto[0])  
texto = "Python"  
print(len(texto))
```

Recorrer una cadena:

```
for letra in texto:  
    print(letra)
```

Convertir a mayúsculas y minúsculas:

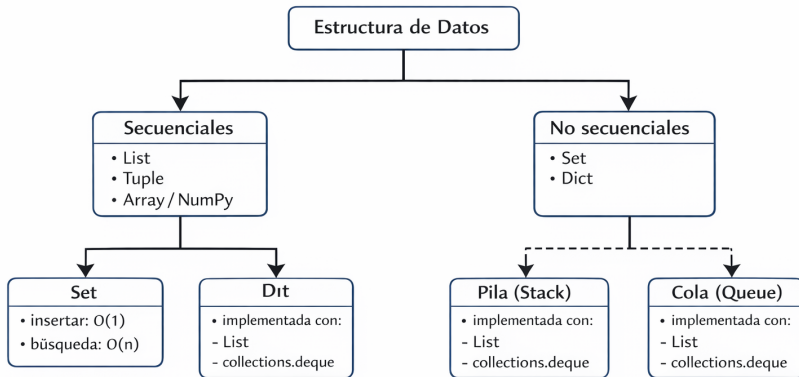
```
print(texto.upper())  
print(texto.lower())
```

Reemplazar palabras:

```
frase = "Hola mundo"  
print(frase.replace("mundo", "Python"))
```

Estructuras de datos

Estructuras de datos en Python (UML conceptual)



Estructuras de datos en Python

Las estructuras de datos permiten almacenar y organizar información.

Principales estructuras:

- Listas
- Tuplas
- Arreglos (arrays)

Son fundamentales para implementar algoritmos.

Listas en Python

Colección ordenada que pueden contener cualquier tipo de dato: números, cadenas, booleanos, ... y también listas.

```
numeros = [10, 20, 30, 40]
```

```
print(numeros[0])  
numeros.append(50)  
numeros.remove(20)
```

```
l = ["una lista", [1, 2]]  
var = l[1][0]
```

Características:

- Permiten elementos de distintos tipos
- Se pueden modificar

Métodos de listas en Python

`L.append( objeto )`
`L.count( valor )`
`L.extend( iterable )`
`L.index( valor )`
`L.insert( indice , objeto )`
`L.pop( indice )`
`L.remove( valor )`
`L.reverse ( )`
`L.sort ( )`

- `append()`: agrega un elemento al final.
- `count()`: cuenta cuántas veces aparece un valor.
- `extend()`: agrega elementos de otra colección.
- `index()`: devuelve la posición de un valor.
- `insert()`: inserta un elemento en una posición.
- `pop()`: elimina y retorna un elemento.
- `remove()`: elimina la primera ocurrencia.
- `reverse()`: invierte el orden de la lista.
- `sort()`: ordena la lista.

Listas en Python

Slicing o Particionado: Consiste en seleccionar porciones de la lista.
(inicio:fin) Python interpretará que queremos una lista que vaya desde la posición inicio a la posición fin, sin incluir este último.

(inicio:fin:salto) en lugar de dos, el tercero se utiliza para determinar cada cuantas posiciones añadir un elemento a la lista.

```
l = [99, True, "una lista", [1, 2]]  
mi_var = l[0:2]      \# mi_var vale [99, True]  
mi_var = l[0:4:2]    \# mi_var vale [99, "una lista"]
```

Tuplas en Python

- Una tupla es una colección de elementos ordenados e inmutables.
- Se define usando paréntesis: ()
- Puede contener diferentes tipos de datos.

Ejemplo:

```
tupla = (10, "Hola", 3.14)
print(tupla)
```

- No se pueden modificar después de crearse.
- Son útiles para almacenar datos constantes.

Operaciones con Tuplas

Acceso a elementos:

```
tupla = (5, 8, 12)  
print(tupla[0])
```

Recorrer una tupla:

```
for elemento in tupla:  
    print(elemento)
```

Ventajas:

- Más rápidas que las listas.
- Protegen la información (no se modifican).
- Útiles para retornar varios valores en funciones.

Arreglos en Python

¿Qué es un arreglo?

- Estructura de datos que almacena elementos del mismo tipo.
- Permite trabajar eficientemente con datos numéricos.
- Se usa en cálculos científicos, ingeniería y análisis de datos.

Arreglo con NumPy:

```
import numpy as np
```

```
arreglo = np.array([1, 2, 3, 4, 5])  
print(arreglo)
```

- Los datos se almacenan de forma ordenada.
- Se accede a ellos por posición (índice).

Operaciones con arreglos

Acceso a elementos:

```
import numpy as np

arreglo = np.array([10, 20, 30, 40])

print(arreglo[0])
print(arreglo[2])
```

Operaciones matemáticas:

```
print(arreglo + 5)
print(arreglo * 2)
print(arreglo.mean())
```

- Ideales para cálculos numéricos.
- Más eficientes que las listas en operaciones matemáticas.

Arreglos multidimensionales en Python

Ejemplo de matriz (2 dimensiones):

```
matriz = np.array([  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9]  
])
```

```
print(matriz[0][1])    # fila 0, columna 1
```

Interpretación:

- Primera posición → fila.
- Segunda posición → columna.
- Representa datos organizados como una tabla.

Uso de estructuras de datos secuenciales en Python

1. Lista (mutable)

```
notas = [3.5, 4.2, 4.8]
notas.append(5.0)
print(notas)
```

2. Tupla (inmutable)

```
coordenadas = (10, 20)
print("Posición X:", coordenadas[0])
```

3. Arreglo numérico (NumPy)

```
import numpy as np
temperaturas = np.array([18, 20, 22, 19])
print(temperaturas.mean())
```

- Guardan datos ordenados.
- Permiten recorrer información secuencialmente.
- Se usan en registros, mediciones y procesamiento de datos.

Algoritmos de Búsqueda

Permiten encontrar elementos dentro de una estructura.

- 1 Secuencial
- 2 Binaria

Ejemplo

Búsqueda secuencial (lineal)

Revisa los elementos uno por uno hasta encontrar el dato.

Características:

- Funciona con listas ordenadas o desordenadas.
- Fácil de implementar.
- Puede ser lenta si hay muchos datos.

Ejemplo en Python:

```
lista = [4, 7, 2, 9, 5]  
buscar = 9
```

```
for i in range(len(lista)):  
    if lista[i] == buscar:  
        print("Encontrado en posición:", i)
```

Búsqueda binaria

Dividir la lista en mitades hasta encontrar el elemento.

- La lista debe estar ordenada.
- Mucho más rápida que la búsqueda secuencial.
- Reduce el número de comparaciones.

```
lista = [2, 4, 5, 7, 9, 12]
buscar = 7
inicio = 0
fin = len(lista) - 1
while inicio <= fin:
    medio = (inicio + fin) // 2
    if lista[medio] == buscar:
        print("Encontrado en posición:", medio)
        break
    elif lista[medio] < buscar:
        inicio = medio + 1
    else:
        fin = medio - 1
```

Algoritmos de ordenamiento

- 1 burbuja —Burbuja
- 2 selección —Selección
- 3 Inserción —Inserción
- 4 Dividir y unir (Merge-sort)

Ejercicios: Cadenas, Listas, Tuplas, Arreglos

1. Cadenas de texto:

```
texto = "Programación"
# a) Imprime la primera letra
# b) Imprime los últimos 3 caracteres
# c) Imprime el texto en mayúsculas
# d) Reemplaza "ción" por "ción Python"
```

2. Listas:

```
notas = [3.5, 4.2, 4.8, 3.9]
# a) Agrega una nueva nota 5.0 al final
# b) Elimina la nota 4.2
#c) Invierte el orden de la lista
```

3. Tuplas:

```
coordenadas = (10, 20, 30)
# a) Imprime la coordenada Y (segunda posición)
# b) Intenta modificar la coordenada Z a 50
```

Ejercicios: Arreglos y Búsqueda

4. Arreglos NumPy:

```
import numpy as np
temperaturas = np.array([18, 20, 22, 19, 21])
# a) Imprime la temperatura máxima
# b) Suma 2 grados a todas las temperaturas
# c) Calcula la temperatura promedio
```

5. Arreglos multidimensionales:

```
matriz = np.array([
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]])
# a) Imprime el elemento fila 2, columna 1
# b) Suma 10 a todos los elementos
# c) Calcula la suma total de todos los elementos
```

Ejercicios Resueltos

1. Cadenas de texto:

```
texto = "Programación"
print(texto[0])          # P
print(texto[-3:])        # ción
print(texto.upper())     # PROGRAMACIÓN
print(texto.replace("ción", "ción Python"))
# Programación Python
```

2. Listas:

```
notas = [3.5, 4.2, 4.8, 3.9]
notas.append(5.0)        # [3.5, 4.2, 4.8, 3.9, 5.0]
notas.remove(4.2)        # [3.5, 4.8, 3.9, 5.0]
notas.reverse()          # [5.0, 3.9, 4.8, 3.5]
```

3. Tuplas:

```
coordenadas = (10, 20, 30)
print(coordenadas[1])    # 20
# coordenadas[2] = 50    # Error: no se puede modificar
```

Ejercicios Resueltos

4. Arreglos NumPy:

```
import numpy as np
temperaturas = np.array([18, 20, 22, 19, 21])
print(temperaturas.max())           # 22
print(temperaturas + 2)             # [20 22 24 21 23]
print(temperaturas.mean())          # 20.0
```

5. Arreglos multidimensionales:

```
matriz = np.array([
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
])
print(matriz[1, 0])                 # 4
print(matriz + 10)                  # [[11 12 13], [14 15 16]
print(matriz.sum())                 # 45
```